CHUYÊN ĐỀ
KĨ THUẬT DỜI GỐC CÂY (REROOTING TECHNIQUE)

Yêu cầu kiến thức trước

Để tiếp cận chuyên đề này, học sinh cần nắm vững các kiến thức sau:

- **Đệ quy và thuật toán DFS (Depth First Search)**.
- **Cấu trúc dữ liệu cây** và các phương pháp **duyệt cây**.
- **Quy hoạch động (Dynamic Programming)**, đặc biệt là **quy hoạch động trên cây**.

I. ĐẶT VẤN ĐỀ

Trong nhiều bài toán trên **cây**, ta cần tính một giá trị nào đó cho toàn bộ cây khi **chọn một đỉnh làm gốc**. Ví dụ:

- Tổng khoảng cách từ một đỉnh đến tất cả các đỉnh khác.
- Giá trị tối ưu của một hàm nào đó trên cây khi chọn gốc tại một đỉnh.
- Giá trị của một cấu trúc con phụ thuộc vào vị trí gốc.

Một cách tiếp cận đơn giản là:

- Lần lượt **chọn mỗi đỉnh i ($1 \le i \le n$) làm gốc**.
- Với mỗi lựa chọn gốc, ta **duyệt cây bằng DFS** để tính giá trị cần tìm.

Khi đó, mỗi lần duyệt cây tốn **O(n)** thời gian, và ta phải thực hiện **n lần**, do đó tổng thời gian là:$O(n^2)$ với **n lớn (ví dụ $n \le 2 \cdot 10^5$)** thì cách làm này không còn khả thi.

Ý tưởng tối ưu

Trong nhiều bài toán, khi ta đã biết kết quả khi **gốc của cây là u**, ta có thể **suy ra nhanh kết quả khi gốc chuyển sang một đỉnh kề v của u**.

Kĩ thuật này được gọi là:

Kĩ thuật dời gốc cây (Rerooting Technique).

Ý tưởng chính:

- Ban đầu chọn một đỉnh (thường là **đỉnh 1**) làm gốc.
- Duyệt cây một lần bằng **DFS** để tính các giá trị ban đầu.
- Sau đó **dịch chuyển gốc từ u sang v** (v là con của u) và **cập nhật lại giá trị trong O(1)**.
- Tiếp tục dời gốc lần lượt qua các đỉnh.

Nhờ đó:

- Mỗi cạnh của cây chỉ được xử lý **một số lần hằng số**.
- Tổng thời gian của thuật toán chỉ còn:

O(n)

Đây là một kĩ thuật rất quan trọng trong các bài toán **DP trên cây nâng cao**, thường xuất hiện trong các kì thi lập trình như **Codeforces, ICPC, VOI, OI**.

II. MÔ HÌNH TỔNG QUÁT

Giả sử ta cần tính một đại lượng **F[u]**, biểu diễn giá trị của cây khi **đỉnh u được chọn làm gốc**.

Cách làm thông thường

Ta có thể viết hàm DFS như sau:

```
DFS(u):
   for mỗi đỉnh v là con của u:
```

```
        DFS(v)
        cập nhật F[u] dựa vào F[v]
```
Sau đó thử tất cả các gốc:
```
xuli():
    maxF = -∞
    for i từ 1 đến n:
        DFS(i)
        maxF = max(maxF, F[i])
    in maxF
```
Phương pháp này có độ phức tạp:
$O(n^2)$
vì mỗi lần chọn gốc ta phải duyệt lại toàn bộ cây.

Áp dụng kĩ thuật dời gốc
Với kĩ thuật **rerooting**, ta thực hiện theo hai bước:
Bước 1: DFS lần thứ nhất
Chọn một đỉnh (thường là **1**) làm gốc ban đầu.
Duyệt DFS để tính các giá trị cho **cây con**:
```
DFS(u, parent):
    for mỗi v kề u:
        nếu v ≠ parent:
            DFS(v, u)
            cập nhật F[u] từ F[v]
```
Sau bước này ta có các thông tin của **mọi cây con**.

Bước 2: Dời gốc (Reroot)
Ta viết thêm một hàm:
reroot(u, parent)
Khi đang coi **u là gốc**, ta thử **dời gốc sang v (con của u)**.
Khi đó ta **cập nhật lại giá trị F[v] từ F[u]** chỉ trong **O(1)** thời gian.
Sau khi xử lý xong nhánh v, ta **khôi phục lại trạng thái ban đầu** rồi tiếp tục sang nhánh khác.
Ý tưởng:
```
reroot(u, parent):
    cập nhật đáp án với F[u]

    for mỗi v kề u:
        nếu v ≠ parent:
            chuyển gốc từ u sang v
            reroot(v, u)
            khôi phục trạng thái ban đầu
```
Nhờ vậy:
- Mỗi cạnh chỉ xử lý **hai lần**.
- Độ phức tạp toàn bộ thuật toán:

O(n)
Ý nghĩa của kĩ thuật dời gốc

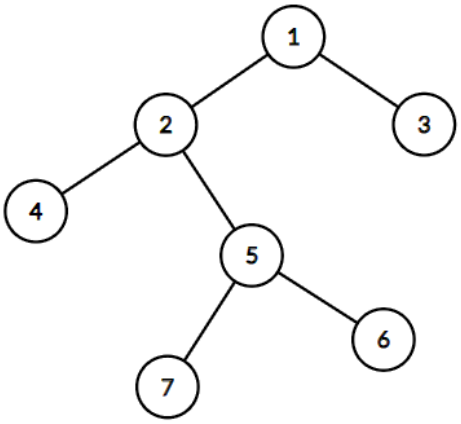
Kĩ thuật rerooting được dùng để giải rất nhiều dạng bài như:

- Tổng khoảng cách trên cây
- Đếm số đỉnh thỏa mãn điều kiện
- DP chọn màu trên cây
- DP subtree + outside subtree
- Tính giá trị tối ưu khi đổi gốc

Đây là một trong những **kĩ thuật quan trọng nhất của DP trên cây**.

1. Bài toán 1: TỔNG ĐƯỜNG ĐI

Cho một cây đồ thị vô hướng gồm N đỉnh.



Yêu cầu: Với mỗi nút u $(1 \le u \le N)$, bạn hãy tính tổng đường đi từ gốc u đến mọi đỉnh còn lại trong cây.

Dữ liệu: Vào từ file văn bản DFROOT.INP:

- Dòng đầu tiên chứa một số nguyên N $(1 \le N \le 10^6)$.
- $N - 1$ dòng tiếp theo, mỗi dòng chứa hai số nguyên u, v $(1 \le u, v \le N, u \ne v)$ thể hiện cho một cạnh trong cây. Mỗi cặp số nguyên u, v chỉ xuất hiện một lần.

Kết quả: Đưa ra file văn bản DFROOT.OUT gồm N dòng, dòng thứ u $(1 \le u \le N)$ là tổng đường đi từ gốc u đến mọi đỉnh trong cây.

Ràng buộc:

- 50% số test ứng với 50% số điểm có $1 \le N \le 5000$.
- 50% số test còn lại ứng với 50% số điểm không có ràng buộc gì thêm.

Ví dụ:

DFROOT.INP	DFROOT.OUT
7 1 2 1 3 2 4 2 5 5 6 5 7	12 9 17 14 10 15 15

Giải thích:

Xem hình vẽ trên.

Xét đỉnh 1, ta có kết quả:

0 + 1 + 1 + 2 + 2 + 3 + 3 = 12;

Xét đỉnh 2, ta có kết quả:

1 + 0 + 2 + 1 + 1 + 2 + 2 = 9;

...

Độ phức tạp thời gian: $O(n^2)$.

Nếu thầy muốn **code "trâu" (brute force) trước** đúng theo phân tích $O(n^2)$trong tài liệu, thì ta **lần lượt lấy mỗi đỉnh làm gốc rồi DFS tính tổng khoảng cách**.

Ý tưởng:

- Với mỗi đỉnh u từ 1 → n
- Chạy DFS(u)
- Tính tổng khoảng cách từ u đến tất cả các đỉnh khác

Độ phức tạp:

$O(n^2)$

Phù hợp **subtask n≤5000**

2. kĩ thuật dời gốc cây

ans[u] tổng khoảng cách từ đỉnh u đến tất cả các đỉnh còn lại trong cây

$$ans[u] = \sum_{v=1}^{n} d(u,v)$$

d(u,v) là **số cạnh trên đường đi giữa u và v**.

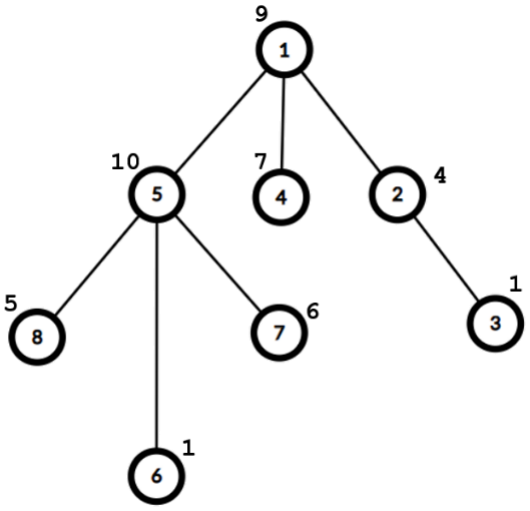
dfs1: Chọn **đỉnh 1 làm gốc ban đầu**.

Ta tính:

- `dep[u]`: độ sâu của đỉnh u
- `sdgoc[u]`: số đỉnh trong **cây con gốc u**

Bài toán 2: CHI PHÍ TRÊN CÂY

Cho một cây gồm N đỉnh. Cây là một đồ thị liên thông vô hướng gồm $N-1$ đỉnh. Với mỗi đỉnh v sẽ có một trọng số a_v.



Gọi $d(u, v)$ là khoảng cách giữa hai đỉnh u và v. Khoảng cách giữa hai đỉnh bất kỳ là số lượng cạnh trên đường đi đơn giữa chúng.

Chi phí trên cây được tính bằng tổng các tích giữa khoảng cách từ đỉnh gốc đến các đỉnh khác với trọng số của đỉnh đó. Hay nói cách khác, nếu ta lấy một đỉnh v bất kỳ làm gốc, thì chi phí trên cây sẽ là: $C = \sum_{i=1}^{N} d(v,i) * a_i$.

Yêu cầu: Tìm giá trị của C_{max} là chi phí trên cây lớn nhất khi chọn một đỉnh v bất kỳ làm gốc.

Dữ liệu: Vào từ file văn bản MAXCOST.INP:

- Dòng đầu tiên chứa một số nguyên N $(1 \le N \le 2 * 10^5)$ là số đỉnh trong cây.
- Dòng thứ hai chứa N số nguyên $a_1, a_2, ..., a_N$ $(1 \le a_i \le 2 * 10^5)$ với a_i là trọng số của đỉnh i.
- $N-1$ dòng tiếp theo, mỗi dòng chứa hai số nguyên u, v $(1 \le u, v \le N, u \ne v)$ thể hiện cho một cạnh trong cây. Mỗi cặp số nguyên u, v chỉ xuất hiện một lần.

Kết quả: Đưa ra file văn bản MAXCOST.OUT một số nguyên duy nhất là C_{max}.

Ví dụ:

MAXCOST.INP	MAXCOST.OUT
8 9 4 1 7 10 1 6 5 1 2 2 3 1 4 1 5 5 6 5 7 5 8	121

Giải thích:

Chọn đỉnh 3 làm gốc, ta có tổng chi phí lớn nhất là: $2 * 9 + 1 * 4 + 0 * 1 + 3 * 7 + 3 * 10 + 4 * 1 + 4 * 6 + 4 * 5 = 18 + 4 + 0 + 21 + 30 + 4 + 24 + 20 = 121$.

Thuật toán:
DFS1
Tính được trọng số tại từng đỉnh.
Và chiều sâu của từng đỉnh
Ban đầu trongso[u]=a[u] → trọng số của đỉnh u là chính a[u].
Xét các đỉnh kề với u, là đỉnh v
Dep[v]=dep[u]+1.
Sau đó dfs(v,u)
Sau khi chạy đến nút lá
trongso[u] += trongso[v]; // tổng trongso nut cha u và các nút con của u
ta tính luôn chi phí cost[1]= cost[1]+dep[u]*a[u]
bước 2: DFS2
bước này kĩ thuật dời gốc
vì lấy max nên kq=max(kq,cost[u])
ta nhận thấy như sau: giả sử khi thay đổi nút cha từ u xuống v thì chi phí từ nhánh u→v giảm đi 1.
Ngược lại chi phí xa v sẽ tăng lên 1
For(v:ke[u])
If(v==parent) continue;
Cost[v]= cost[u] -trongso[v] +tongtt -trongso[v]
Dfs2(v,u)
Trong đó tongtt=tong+a[i].

Bài toán 3: CÂY CON ĐEN TRẮNG

Cho một cây đồ thị vô hướng gồm N đỉnh. Mỗi đỉnh v có màu trắng hoặc đen, $a_v = 0$ nếu đỉnh v có màu đen, và $a_v = 1$ nếu đỉnh v có màu trắng.

Cây con của cây là một đồ thị con liên thông có trong cây đã cho.

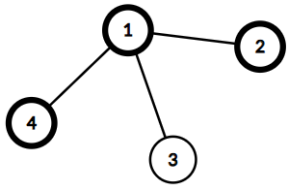
Yêu cầu: Với mỗi đỉnh v $(1 \le v \le N)$, trong tất cả các cây con của cây có chứa đỉnh v, bạn hãy tìm giá trị lớn nhất của $c_w - c_b$ với c_w, c_b lần lượt là số lượng đỉnh trắng và đỉnh đen của một cây con.

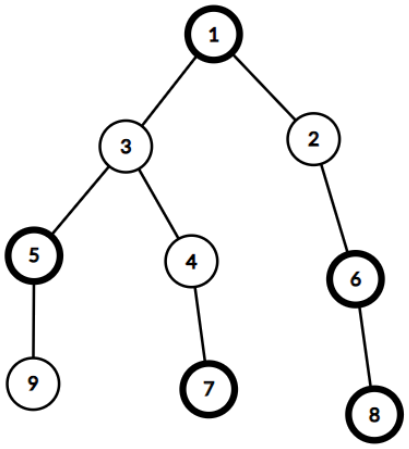
Dữ liệu: Vào từ file văn bản BWTREE.INP:

- Dòng đầu tiên chứa một số nguyên N $(2 \le N \le 2 \times 10^5)$.
- Dòng thứ hai chứa N số nguyên $a_1, a_2, ..., a_N$ $(0 \le a_i \le 1)$ với a_i biểu thị màu sắc của đỉnh i trong cây.
- $N - 1$ dòng tiếp theo, mỗi dòng chứa hai số nguyên u, v $(1 \le u, v \le N, u \ne v)$ thể hiện cho một cạnh trong cây. Mỗi cặp số nguyên u, v chỉ xuất hiện một lần.

Kết quả: Đưa ra file văn bản BWTREE.OUT gồm N số $r_1, r_2, ..., r_N$ với r_i là chênh lệch lớn nhất giữa đỉnh trắng và đỉnh đen trong tất cả các cây con chứa đỉnh i. Mỗi số được viết trên một dòng.

Ví dụ:

BWTREE.INP	BWTREE.OUT	Giải thích:
4 0 0 1 0 1 2 1 3 1 4	0 -1 1 -1	

BWTREE.INP	BWTREE.OUT	Giải thích:
9 0 1 1 1 0 0 0 0 1 1 2 1 3 3 4 3 5 2 6 4 7 6 8 5 9	2 2 2 2 2 1 1 0 2	

. Bài toán 5: TÔ MÀU CÂY

https://codeforces.com/contest/1187/problem/E

Cho một cây đồ thị vô hướng gồm N đỉnh. Bạn sẽ thực hiện một trò chơi như sau:

- Ban đầu, tất cả các đỉnh đều có màu trắng.
- Ở lượt đầu tiên, bạn sẽ chọn một đỉnh bất kỳ và tô đỉnh đó thành màu đen.
- Sau đó, ở những lượt tiếp theo bạn sẽ chọn một đỉnh trắng mà kề với đỉnh đen bất kỳ (liên thuộc bằng một cạnh) và tô đỉnh đó thành màu đen.
- Trò chơi sẽ kết thúc khi toàn bộ N đỉnh đều có màu đen.

Với mỗi lần bạn chọn một đỉnh để tô, bạn sẽ nhận được số điểm bằng số lượng đỉnh của thành phần liên thông chỉ gồm các điểm trắng có chứa đỉnh đã chọn.

Yêu cầu: Hãy cho biết bạn có thể đạt được nhiều nhất bao nhiêu điểm trong trò chơi này.

Dữ liệu: Vào từ file văn bản TPAINT.INP:

- Dòng đầu tiên chứa một số nguyên N $(2 \leq N \leq 2 \times 10^5)$.
- $N - 1$ dòng tiếp theo, mỗi dòng chứa hai số nguyên u, v $(1 \leq u, v \leq N, u \neq v)$ thể hiện cho một cạnh trong cây. Mỗi cặp số nguyên u, v chỉ xuất hiện một lần.

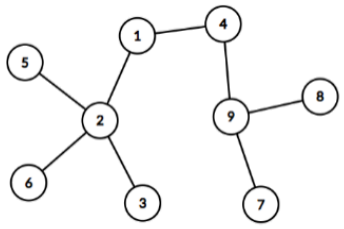
Kết quả: Đưa ra file văn bản TPAINT.OUT một số nguyên duy nhất là số điểm lớn nhất có thể đạt được.

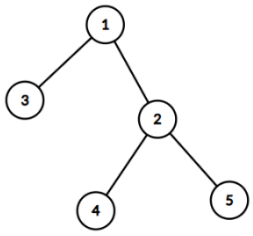
Ví dụ:

TPAINT.INP	TPAINT.OUT
9 1 2 2 3 2 5 2 6 1 4 4 9 9 7 9 8	36
5	14

TEST 1:

Giải thích:



1 2 1 3 2 4 2 5		Thứ tự chọn các đỉnh để tô màu: 7, 9, 8, 4, 1, 2, 3, 5, 6. Tổng điểm là: $9 + 8 + 1 + 6 + 5 + 4 + 1 + 1 + 1 = 36$ điểm. ***TEST 2:*** Thứ tự chọn các đỉnh để tô màu: 3, 1, 2, 4, 5. Tổng điểm là: $5 + 4 + 3 + 1 + 1 = 14$ điểm. 

Phân tích thuật toán:

Ta lần lượt chọn điểm đầu tiên là u (với u=1..n) để tìm điểm chơi thu được lớn nhất. Giả sử ta chọn đỉnh đầu tiên là u. Xét cây gốc u, ta chọn lần lượt đỉnh u rồi tới các đỉnh con của u,… Quy luật là luôn chọn đỉnh cha trước rồi mới tới chọn đỉnh con. Gọi F[i] là số điểm thu được khi chọn đỉnh i và các đỉnh trong cây con gốc i. Ta tìm cách tính F[i] như sau.

Trước khi chọn nút i thì:

- Nút cha của i có màu đen;
- Các nút thuộc cây con gốc i đều có màu trắng;

Khi chọn nút i, ta được số điểm là số điểm trắng liên thông với đỉnh i, chính là số nút của cây con gốc i, kí hiệu SIZE[i]. Tiếp đến là chọn các nút con của i. Thứ tự chọn các nút con của i là không quan trọng. Nếu chọn nút con j của i, ta sẽ được số điểm là F[j]. Vậy:

$$F[i] = SIZE[i] + \sum F[j], \text{ j là con của i.}$$

Vận dụng kĩ thuật dời gốc cây để dời gốc cây từ u xuống nút con v của u. Ta có các thay đổi sau:

Pha 1: Cắt quan hệ cha con u-v:

- `F[u]-=SIZE[v]+F[v];`

- `SIZE[u]-=SIZE[v];`

Pha 2: Thêm quan hệ cha con v-u:

- `F[v]+=SIZE[u]+F[u];`
- `SIZE[v]+=SIZE[u];`

IV. CÁC BÀI TOÁN TỰ LUYỆN THÊM

1. Bài toán 6: CẢI TẠO ĐƯỜNG

https://codeforces.com/contest/543/problem/D

Đất nước XYZ có n thành phố và $n - 1$ con đường hai chiều và có thể đi từ mọi thành phố đến bất kỳ một thành phố nào khác nếu chỉ di chuyển dọc theo các con đường. Các thành phố lần lượt được đánh số từ 1 đến n.

Ban đầu tất cả các con đường đều không tốt, nhưng chính phủ muốn cải thiện tình trạng của một số con đường. Các người dân sẽ hài lòng về việc cải thiện đường xá nếu như con đường từ thủ đô nằm tại thành phố x đến bất kỳ thành phố nào khác có nhiều nhất một con đường không tốt.

Yêu cầu: Với mỗi trường hợp thủ đô nằm tại thành phố x ($1 \le x \le n$) bạn hãy giúp chính phủ xác định số lượng cách để cải thiện một số con đường để đáp ứng được sự hài lòng từ người dân. Vì kết quả có thể rất lớn nên bạn hãy in ra số dư kết quả của bài toán khi chia cho 1000000007 ($10^9 + 7$).

Dữ liệu vào: *Từ file văn bản ROADS.INP có cấu trúc:*

- Dòng thứ nhất chứa một số nguyên n ($1 \le n \le 2 \times 10^5$) là số lượng thành phố có trong đất nước XYZ.
- Dòng thứ hai chứa $n - 1$ số nguyên $p_2, p_3, ..., p_n$ ($1 \le p_i < i$), mỗi số p_i mô tả rằng đất nước có một con đường hai chiều giữa thành phố p_i và thành phố i.

Kết quả ra: *Ghi ra file văn bản ROADS.OUT có cấu trúc:*

- Gồm n số nguyên trên một dòng, số thứ i là số dư số lượng cách để cải thiện một số con đường khi thủ đô nằm ở thành phố i khi chia cho 1000000007 ($10^9 + 7$).

Ví dụ:

ROADS.INP	ROADS.OUT
3 1 1	4 3 3
5 1 2 3 4	5 8 9 8 5

Giải thích: Ví dụ 1, ta có đồ thị như sau:

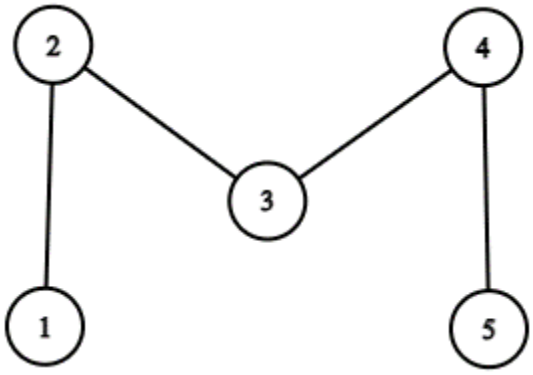
Nếu đặt thủ đô tại đỉnh 1 thì có 4 cách sửa đường:

- C1: không sửa đường nào;
- C2: Sửa đường 1-2;
- C3: Sửa đường 1-3;
- C4: Sửa cả 2 đường 1-2 và 1-3;

Nếu đặt thủ đô tại đỉnh 2 (hoặc đỉnh 3) thì có 3 cách sửa đường:

- C1: Sửa đường 1-2;
- C2: Sửa đường 1-3;
- C3: Sửa cả 2 đường 1-2 và 1-3;

Ví dụ 2, ta có đồ thị như sau:

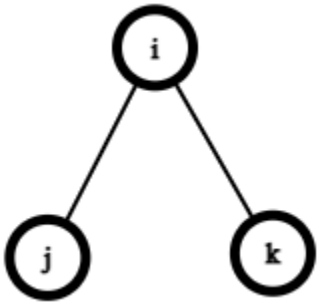


Để đường đi từ 1 đến 5 không có quá 1 đường xấu thì ta phải chọn ít nhất 3 đoạn đường để sửa: nếu chọn 3 đoạn thì số cách là $C_4^3 = 4$, nếu chọn 4 đoạn thì $C_4^4 = 1$. Suy ra, nếu chọn 1 hoặc 5 làm thủ đô thì số cách sửa là 5 cách.

Nếu chọn đỉnh 3 làm thủ đô thì 2 đường đi 3-2-1 và 3-4-5 phải sửa độc lập, mỗi đường gồm 2 đoạn nên có 3 cách sửa, suy ra số cách sửa đường là 3*3=9 cách.

Phân tích thuật toán:

Cho một đỉnh u làm gốc, ta sẽ tính số cách sửa đường khi u là thủ đô. Gọi F[i] là số cách sửa đường trong cây con gốc i thỏa mãn yêu cầu đề bài (các đường đi từ i đến các nút x trong cây con gốc x có không quá 1 đường xấu). Ta tìm cách tính F[i]:



Giải sử j và k là các nút con của i. Cách sửa đường từ i qua j sẽ độc lập với đường đi từ i qua k, nên ta áp dụng quy tắc nhân.

Xét các đường đi từ i qua j rồi đi tiếp đến đỉnh x. Có 2 trường hợp xảy ra:

TH1: Đoạn i-j được sửa lại, khi đó, ta phải sửa tiếp để từ j đến x có không quá 1 đoạn xấu. Số cách sửa là F[j].

TH2: Đoạn i-j không được sửa, do đó, các đoạn khác trong cây con gốc j phải được sửa. Ta có số cách là 1.

Vậy, các đường đi từ i qua hướng j có F[j]+1 cách. Tương tự cho các đường đi qua nút con j' khác của i. Ta có công thức:

$$F[i] = \prod(F[j]+1), \text{ với mọi j là con của i.}$$

Xét thao tác dời gốc cây từ u xuống nút con v của u:

Pha 1: Hủy bỏ quan hệ cha con u-v: `F[u] /= (F[v]+1);`

Pha 2: Thêm quan hệ cha con v-u: `F[v] *= (F[u]+1);`

Ta cần xử lí thêm thao tác với phần dư mod.

2. Bài toán 7: ĐÁNH SỐ TRÊN CÂY

https://atcoder.jp/contests/abc160/tasks/abc160_f

Cho một cây đồ thị vô hướng gồm N đỉnh và $N - 1$ cạnh. Cạnh thứ i trong cây nối liền giữa hai đỉnh a_i và b_i.

Với mỗi số nguyên $k = 1, 2, ..., N$ bạn hãy thực hiện các thao tác sau:

- Đầu tiên, đánh số 1 tại đỉnh k.
- Sau đó, khi đánh số các số từ 2 đến N bạn phải chọn ra một đỉnh không được đánh số mà kề với đỉnh đã được đánh số.
- Gọi r_k là số cách đánh số thỏa mãn với số 1 được đánh số tại đỉnh k.

Yêu cầu: Bạn hãy xác định dãy gồm N số $r_1, r_2, ..., r_N$ được mô tả ở trên. Kết quả của mỗi số r_k rất lớn nên bạn hãy in ra $r_k \bmod 10^9 + 7$ với $a \bmod b$ là số dư khi chia a cho b.

Dữ liệu vào: *Từ file văn bản DISTRIBUTE.INP có cấu trúc:*

- Dòng thứ nhất chứa một số nguyên N ($1 \le N \le 2 \times 10^5$) là số lượng đỉnh trong cây.
- $N - 1$ dòng tiếp theo, mỗi dòng chứa hai số nguyên a_i, b_i ($1 \le a_i, b_i \le N, a_i \ne b_i$) mô tả một cạnh trong cây.

Kết quả ra: *Ghi ra file văn bản DISTRIBUTE.OUT có cấu trúc:*

- Gồm N dòng, dòng thứ k ($1 \le k \le N$) in ra $r_k \bmod 10^9 + 7$ là số dư số lượng cách đánh số thỏa mãn khi đỉnh k được đánh số 1 khi chia cho $10^9 + 7$.

Ví dụ 1:

DISTRIBUTE.INP	DISTRIBUTE.OUT
3 1 2 1 3	2 1 1
2 1 2	1 1
5 1 2 2 3	2 8 12

3 4 3 5	3 3
8 1 2 2 3 3 4 3 5 3 6 6 7 6 8	40 280 840 120 120 504 72 72

Gợi ý:

Cách đánh số sẽ tương tự cách tô màu ở bài toán 5 (TPAINT – Tô màu trên cây): đánh số nút cha trước, nút con sau.

Công thức: F[i] = (SIZE[i]-1)! $\prod \frac{F[j]}{SIZE[j]!}$, với j là nút con của i.

Dời gốc cây từ nút u xuống nút con v:

Pha 1:

- `F[u] /= (SIZE[u]-1)! * F[v]/SIZE[v]!`
- `SIZE[u]-=SIZE[v]`
- `F[u] *= (SIZE[u]-1)!`

Pha 2:

- `F[v] /= (SIZE[v]-1)!`
- `SIZE[v]+=SIZE[u];`
- `F[v] *= (SIZE[v]-1)! * F[u]/SIZE[u]!`

3. Bài toán 8: SUBTREE

https://atcoder.jp/contests/dp/tasks/dp_v

Cây là đồ thị vô hướng, liên thông và không có chu trình. Một cây N đỉnh sẽ có (N−1) cạnh. Cho một cây có N đỉnh, các đỉnh được đánh số 1, 2, ..., N. Cạnh thứ i ($1 \le i \le N-1$) kết nối 2 đỉnh x_i và y_i.

Taro quyết định tô mỗi đỉnh của cây bởi một trong hai màu trắng hoặc đen, sao cho với hai đỉnh được tô đen bất kì, tất cả các đỉnh trên đường đi giữa hai đỉnh đó cũng được tô đen.

Bạn được cho một số nguyên dương M. Với mỗi v ($1 \le v \le N$), hãy trả lời câu hỏi dưới đây:

Giả sử đỉnh v đã được tô màu đen, có bao nhiêu cách tô (N−1) đỉnh còn lại để thỏa mãn điều kiện trên? Do đáp án có thể rất lớn, in ra số dư của kết quả chia cho M.

Dữ liệu vào: vài từ tệp SUBTREE.INP:

- Dòng đầu tiên chứa 2 số N và M là số đỉnh và modulo.
- Dòng thứ i trong (N−1) dòng tiếp theo chứa 2 số x_i và y_i là 2 đỉnh được nối bởi cạnh thứ i.

Kết quả ra: ghi ra tệp SUBTREE.OUT

- In ra N dòng, dòng thứ i in ra đáp án cho câu hỏi trên với đỉnh i theo modulo M.

Giới hạn

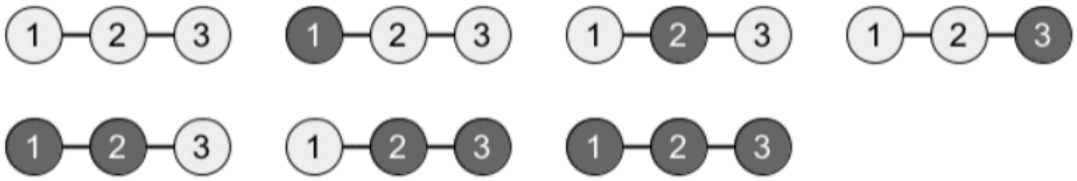
- Tất cả các giá trị trong input là số nguyên.
- $1 \le N \le 10^5$
- $2 \le M \le 10^9$ (lưu ý rằng M không nhất thiết phải là số nguyên tố)
- $1 \le x_i, y_i \le N$ ($1 \le i \le N-1$)
- Đồ thị đã cho là cây.

Ví dụ:

SUBTREE.INP	SUBTREE.OUT
3 100 1 2	3 4

2 3	3
4 100 1 2 1 3 1 4	8 5 5 5
1 100	1
10 2 8 5 10 8 6 5 1 5 4 8 2 10 3 6 9 2 1 7	0 0 1 1 1 0 1 0 1 1

Giải thích ví dụ thứ nhất



Trên đây là tất cả những cách tô màu thỏa mãn điều kiện đề bài. (Cách tô đỉnh 1 và 3 đen không thỏa mãn do đồ thị tạo bởi 2 đỉnh 1 và 3 không liên thông)

Trong 7 cách trên, có

- 3 cách có đỉnh 1 được tô màu đen
- 4 cách có đỉnh 2 được tô màu đen
- 3 cách có đỉnh 3 được tô màu đen

Gợi ý:

Công thức: F[i] = 1 + ∏ F[j], j là nút con của i.

Dời gốc cây từ u xuống nút con v:

- `F[u] = (F[u]-1)/F[v]+1;`
- `F[v] = (F[v]-1)*F[u]+1;`

4. Bài toán 9: CÂY XEN KẼ

https://codeforces.com/contest/960/problem/E

Cho cây có n nút được đánh số từ 1 đến n. Mỗi nút có một giá trị W_i.

Đường đi đơn từ đỉnh u_1 đếm u_m gồm m đỉnh $u_1 \rightarrow u_2 \rightarrow u_3 \rightarrow \ldots \rightarrow u_m$ có một hàm xen kẽ $A(u_1, u_m)$ được định nghĩa:

$$A(u_1, u_m) = \sum_{i=1}^{m} (-1)^{i+1}.W_{u_i}$$

Chấp nhận đường đi chỉ có 1 đỉnh (u_1=u_m).

Yêu cầu: Tính tổng của các hàm xen kẽ của tất cả các đường đi đơn. Hai đường đi đơn khác nhau nếu điểm xuất phát khác nhau hoặc điểm kết thúc khác nhau. Lấy phần dư của kết quả chia cho 10^9+7.

Dữ liệu vào:

- Dòng đầu chứa số n ($2 \le n \le 2.10^5$) là số đỉnh của cây.
- Dòng thứ hai chứa n số $W_1, W_2, \ldots, W_n$ là giá trị tại các đỉnh.
- n-1 dòng tiếp theo, mỗi dòng chứa 2 số nguyên u và v cho biết có cạnh nối hai đỉnh u và v ($1 \le u, v \le n, u \ne v$).

Kết quả ra: Ghi ra tổng các hàm xen kẽ của tất cả đường đi đơn chia lấy dư cho 10^9+7.

Ví dụ:

INPUT	OUTPUT
4 -4 1 5 -2 1 2 1 3 1 4	40
8 -2 6 -4 -4 -9 -3 -7 23 8 2	4

2 3 1 4 6 5 7 6 4 7 5 8	

Giải thích ví dụ 1:

Đường đi đơn từ 1 đến 2: $A(1,2)=1\cdot(-4)+(-1)\cdot 1=-5$

Đường đi đơn từ 1 đến 3: $A(1,3)=1\cdot(-4)+(-1)\cdot 5=-9$

Đường đi đơn từ 2 đến 4: $2\rightarrow 1\rightarrow 4$: $A(2,4)=1\cdot(1)+(-1)\cdot(-4)+1\cdot(-2)=3$

Đường đi đơn từ 1 đến 1: $A(1,1)=1\cdot(-4)=-4$

Tương tự, $A(2,1)=5$; $A(3,1)=9$; $A(4,2)=3$; $A(1,4)=-2$; $A(4,1)=2$; $A(2,2)=1$; $A(3,3)=5$; $A(4,4)=-2$; $A(3,4)=7$; $A(4,3)=7$; $A(2,3)=10$; $A(3,2)=10$.

Vậy, kết quả là $(-5)+(-9)+3+(-4)+5+9+3+(-2)+2+1+5+(-2)+7+7+10+10=40$

Gợi ý:

Công thức: F[i] = SIZE[i]*V[i] − ∑ F[j], j là con của i;

Dời gốc cây từ u xuống nút con v:

- `F[u] = F[u]-W[u]*SIZE[v] + F[v];`
- `SIZE[u]-=SIZE[v];`
- `F[v] = F[v] + W[v]*SIZE[u] - F[u]`
- `SIZE[v]+=SIZE[u];`

5. Bài toán 10: CỨU NAGATO

Link: https://dmoj.ca/problem/dmopc14c4p6